



Terraform — Complete Notes

Infrastructure as Code · Provision any cloud declaratively

A reader-friendly, all-in-one reference for Terraform.

1. 🛠️ What is Terraform (IaC)

Terraform (by HashiCorp) is the leading **Infrastructure as Code (IaC)** tool. You **declare** your desired infrastructure — servers, networks, databases, DNS — in human-readable config files, and Terraform figures out the API calls to make reality match. It's **declarative** (you describe the *what*, not the *how*) and **cloud-agnostic** (AWS, Azure, GCP, and 1000s of providers).

💡 **Mental model:** Terraform is a **diff engine for infrastructure**. You write the desired end state; it compares that to the **current state** and computes the minimal set of creates/updates/deletes to reconcile them. Same config + same state = same infra (reproducible).

Why it matters: version-controlled, reviewable, repeatable infrastructure; no click-ops drift; easy to spin up/tear down identical environments.

2. 🧩 Core Concepts

Concept	Description
Provider	Plugin for a platform (aws, azure, google, kubernetes...).
Resource	A piece of infra to manage (<code>aws_instance</code> , <code>aws_s3_bucket</code>).
Data source	Read-only lookup of existing infra (<code>data "aws_ami" ...</code>).
State	Terraform's record of what it manages (<code>terraform.tfstate</code>).
Module	A reusable package of <code>.tf</code> files (inputs → resources → outputs).
Variable	Parameterized input (<code>var.region</code>).
Output	A value exported from a config/module.
Plan	The preview of changes Terraform will make.
Backend	Where state is stored (local, S3, Terraform Cloud...).

3. HCL Syntax Basics

Terraform configs use **HCL (HashiCorp Configuration Language)** in `.tf` files:

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = var.region
}

resource "aws_instance" "web" {
  ami          = data.aws_ami.al2.id
  instance_type = "t3.micro"
  tags = {
    Name = "web-${var.env}"
  }
}
```

- **Block** = `type "label" "name" { ... }`. Reference an attribute as `aws_instance.web.id`.
- Strings interpolate with `${ }`; supports expressions, functions (`join` , `lookup` , `cidrsubnet`), `for` loops, and conditionals.
- **count** and **for_each** create multiple instances of a resource.

4. The Core Workflow

The day-to-day Terraform cycle:

```
terraform init    → terraform plan    → terraform apply    → terraform destroy
download providers  preview the diff      make it real          tear it all down
+ set up backend    (create/update/delete) (asks to confirm)
```

Command	Does
init	Download providers/modules, configure the backend. Run first (and after adding providers/modules).
plan	Show what will change — dry run , no changes made. Review before applying.
apply	Execute the plan to create/update/destroy resources (prompts to confirm).
destroy	Remove all managed infrastructure.
fmt / validate	Format code / check syntax.

Always **plan** before **apply** — it's the review step that shows exactly what Terraform will create, change, or destroy.

5. 🗄️ State

- Terraform records what it manages in a **state file** (`terraform.tfstate`) — the mapping between your config and real-world resources. It's how Terraform knows what already exists.
- **Never edit state by hand**; use `terraform state` commands. **State can contain secrets** — protect it.
- **Remote state** (S3 + DynamoDB lock, or Terraform Cloud) is essential for teams: shared, versioned, and **locked** so two people can't apply at once.
- **Drift** = real infra changed outside Terraform; `plan` detects it. `terraform import` brings existing resources under management.

```
terraform {
  backend "s3" {
    bucket      = "my-tf-state"
    key         = "prod/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "tf-locks"      # state locking
    encrypt     = true
  }
}
```

⚠️ **State locking** prevents concurrent applies from corrupting state — always use a locking backend for team work.

6. 🛠️ Variables & Outputs

```
variable "region" {
  type    = string
  default = "us-east-1"
}

variable "instance_count" {
  type    = number
  default = 2
}

output "instance_ids" {
  value = aws_instance.web[*].id
}
```

- Set variables via `terraform.tfvars`, `-var`, `-var-file`, or `TF_VAR_*` env vars.
 - **Mark secrets** `sensitive = true` so they're hidden in output (still in state, though).
 - **Outputs** expose values (IPs, ARNs) for users or other modules/configs.
-

7. 🚧 Providers

- A **provider** is a plugin that talks to a platform's API. The **AWS provider** maps `aws_*` resources to AWS API calls; there are providers for Azure, GCP, Kubernetes, GitHub, Datadog, Cloudflare, and 3,000+ more (the **Terraform Registry**).
- Pin provider **versions** (`~> 5.0.0`) for reproducible builds.
- Configure auth per provider (e.g. AWS via env vars, shared credentials, or an IAM role).
- One config can use **multiple providers** (multi-cloud), and provider **aliases** for multiple regions/accounts.

8. 📦 Modules

- A **module** is a reusable container of resources — write once, call many times. The top-level config is the **root module**; it calls **child modules**.

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws" # registry, git, or local path
  version = "5.0.0"
  name    = "prod-vpc"
  cidr    = "10.0.0.0/16"
}

# use its outputs
resource "aws_instance" "web" {
  subnet_id = module.vpc.public_subnets[0]
}
```

- Modules take **inputs** (variables), create resources, and expose **outputs**.
- Sources: **Terraform Registry**, Git repos, or local paths. Modules are the key to **DRY**, consistent infrastructure.

9. 🔗 Dependencies & Data Sources

- Terraform builds a **dependency graph** automatically from references — `subnet_id = aws_subnet.x.id` makes the instance depend on the subnet, so it's created after.
- Use `depends_on` only for hidden dependencies Terraform can't infer.
- **Data sources** read existing infra you don't manage:

```
data "aws_ami" "al2" {
  most_recent = true
  owners      = ["amazon"]
  filter { name = "name" values = ["al2023-ami-*-x86_64"] }
}

# reference: data.aws_ami.al2.id
```

Terraform parallelizes independent resources and orders dependent ones via the graph — you rarely set order manually.

10. 📁 Workspaces & Environments

- **Workspaces** let one config have multiple **separate states** (e.g. `dev`, `staging`, `prod`) — `terraform workspace new prod`.
- They're handy for lightweight separation, but many teams prefer **separate directories/state per environment** + a shared module for stronger isolation.
- Reference the current workspace via `terraform.workspace` to vary names/sizes per env.

11. ⚙️ Provisioners (Use Sparingly)

- **Provisioners** (`remote-exec`, `local-exec`, `file`) run scripts on a resource after creation.
- They're a **last resort** — they break the declarative model, aren't tracked in state well, and make plans less predictable.
- Prefer **cloud-init/user-data**, baked **AMIs (Packer)**, or **config management (Ansible)** over provisioners.

12. ⚖️ Terraform vs CloudFormation vs Ansible

	Terraform	CloudFormation	Ansible
Type	IaC (provisioning)	IaC (provisioning)	Config management
Scope	Multi-cloud	AWS only	Any (agentless, SSH)
Language	HCL (declarative)	YAML/JSON (declarative)	YAML (procedural-ish)
State	Explicit state file	Managed by AWS	Stateless
Best at	Provisioning infra across clouds	Deep AWS-native integration	Installing/configuring software on servers

Terraform vs CloudFormation: both provision infra declaratively; Terraform is multi-cloud with its own state, CloudFormation is AWS-native and AWS-managed. **Terraform vs Ansible:** Terraform *provisions* infrastructure; Ansible *configures* what runs on it (often used together).

13. ✅ Best Practices

- **Always** `plan` before `apply`; review the diff (especially **destroys**).
- **Remote state with locking** (S3 + DynamoDB / Terraform Cloud) for teams; never commit `terraform.tfstate` to Git.
- **Pin provider + module versions**; commit `.terraform.lock.hcl`.
- **Use modules** for reuse and consistency; keep configs **DRY**.
- **Separate state per environment** (dirs or workspaces); parameterize with variables.
- **Don't hard-code secrets** — use variables marked `sensitive`, env vars, or a secrets manager (remember secrets land in state — protect it).
- `fmt` + `validate` in CI; run `plan` on PRs and `apply` only after review.
- Avoid **provisioners**; prefer user-data/Packer/Ansible. Never make changes by hand (causes drift).

14. 🧠 Quick Mental Model

- **Terraform = declarative, multi-cloud Infrastructure as Code** — describe the desired state, it reconciles reality.
 - It's a **diff engine**: desired config vs **state** vs real infra → minimal create/update/delete.
 - Workflow: **init** → **plan** (dry run) → **apply** → **destroy**. Always plan first.
 - **State** maps config ↔ real resources; use **remote state + locking** for teams; protect it (has secrets).
 - **Providers** talk to platforms (`aws_*`); **resources** are managed infra; **data sources** read existing infra.
 - **Modules** = reusable infra packages (inputs → resources → outputs); keep it DRY.
 - Dependencies are inferred from **references** (graph); `depends_on` only for hidden ones.
 - vs **CloudFormation** (AWS-only) and **Ansible** (config management, not provisioning).
-

15. ? Common Interview Questions

Rapid-fire questions interviewers ask about Terraform:

- **Q: What is Terraform / IaC?** — A declarative, multi-cloud tool to define infrastructure as version-controlled code; Terraform reconciles real infra to match your config.
 - **Q: plan vs apply ?** — `plan` is a dry run showing what will change; `apply` actually makes the changes (after confirmation).
 - **Q: What is the state file and why does it matter?** — It maps your config to real resources so Terraform knows what exists; it can hold secrets and must be protected and, for teams, stored remotely with locking.
 - **Q: Why remote state + locking?** — Shared, versioned state for teams, and locking prevents two concurrent applies from corrupting it.
 - **Q: Declarative vs imperative?** — You declare the desired end state; Terraform decides the steps (vs imperative tools where you script each step).
 - **Q: What is a provider?** — A plugin that maps Terraform resources to a platform's API (AWS, Azure, GCP, Kubernetes...).
 - **Q: What is a module?** — A reusable package of Terraform configs with inputs and outputs — the unit of reuse/DRY.
 - **Q: How does Terraform know resource order?** — It builds a dependency graph from references; `depends_on` handles implicit dependencies.
 - **Q: count vs for_each?** — `count` creates N copies by index; `for_each` creates one per item in a map/set (stable keys, easier to add/remove).
 - **Q: Terraform vs CloudFormation?** — Both provision declaratively; Terraform is multi-cloud with its own state, CloudFormation is AWS-only and AWS-managed.
 - **Q: Terraform vs Ansible?** — Terraform provisions infrastructure; Ansible configures software on it — complementary, often used together.
 - **Q: How do you handle secrets?** — Variables marked `sensitive`, env vars, or a secrets manager; never commit them — and protect state since secrets land there.
-

📝 Notes compiled as a quick reference for Terraform.